Gen-AI Hands On-2

NAME: Aakanksha Palavalli

SRN: PES2UG23CS925

Section: A

Documentation :

Part 1: LangChain Foundation

Part1a:

Cell 1

Installs required Python libraries so LangChain and related tools can run.

Cell 2

Loads environment variables (like API keys) so the model can authenticate safely.

API's:

APIs allow our program to communicate with external services like an AI model without building everything from scratch. Using API keys securely ensures only authorized access and protects sensitive information.

Cell 3

Imports the Gemini LLM class so we can create and use the language model.

Cell 4

Defines a sample prompt so we can test how the model responds.

Cell 5

Runs the model with a specific temperature setting to observe behavior.

Temperature setting:

Temperature controls how creative or consistent the model is. A low value gives predictable answers, while a higher value allows more variation

When temperature = 0 or 1

```
from langchain_google_genai import ChatGoogleGenerativeAI

# Model A: The "Accountant" (Precision)
llm_focused = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.0)

# Model B: The "Poet" (Creativity)
llm_creative = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=1.0)
```

7. Experiment: Consistency vs. Creativity

We will ask both models to "Define the word 'Idea' in one sentence." We will run the code **TWICE** for each model.

Hypothesis:

- The Focused model (Temp=0) should say the *exact same thing* both times.
- The Creative model (Temp=1) should say *different things*.

```
prompt = "Define the word 'Idea' in one sentence."

print("--- FOCUSED (Temp=0) ---")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")
```

```
--- FOCUSED (Temp=0) ---
Run 1: An idea is a thought, concept, or suggestion that is formed or exists in the mind.
Run 2: An idea is a thought, concept, or mental image formed in the mind.
```

```
print("--- CREATIVE (Temp=1) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")
```

```
--- CREATIVE (Temp=1) ---
Run 1: An idea is a thought, concept, or plan that originates and exists in the mind as a product of mental activity.
Run 2: An idea is a thought, concept, or mental impression existing in the mind.
```

When temperature = 2

```
from langchain_google_genai import ChatGoogleGenerativeAI

# Model A: The "Accountant" (Precision)
llm_focused = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=2.0)

# Model B: The "Poet" (Creativity)
llm_creative = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=1.0)
```

7. Experiment: Consistency vs. Creativity

We will ask both models to "Define the word 'Idea' in one sentence." We will run the code **TWICE** for each model.

Hypothesis:

- The Focused model (Temp=0) should say the *exact same thing* both times.
- The Creative model (Temp=1) should say *different things*.

```
prompt = "Define the word 'Idea' in one sentence."

print("--- FOCUSED (Temp=0) ---")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")
```

```
--- FOCUSED (Temp=0) ---
Run 1: An idea is a thought, concept, or mental impression formulated in the mind, often representing a plan, suggestion, or understanding.
Run 2: An idea is a thought, concept, or mental impression existing in the mind.
```

```
print("--- CREATIVE (Temp=1) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")
```

```
--- CREATIVE (Temp=1) ---
Run 1: An idea is a thought, concept, or plan conceived in the mind, often as a suggestion, solution, or understanding.
Run 2: An idea is a thought or concept that arises in the mind.
```

The temperature setting in a language model controls how creative or predictable the output is. When the temperature is 0, the model gives very consistent and repeatable answers because it always picks the most likely words. As the temperature increases, the model starts giving more varied and creative responses. At very high temperatures, the answers can change a lot between runs because the model explores less common word choices. Even then, it usually stays meaningful because it is trained to produce sensible text. In short, higher temperature means more creativity, while lower temperature means more consistency.

Part 1b — Prompts & Parsers

Cell 6

Reuses setup from earlier so we can continue working with the model.

Cell 7

Imports message classes used to structure system and user instructions.

Cell 8

Imports the prompt template class used to build dynamic prompts.

Cell 9

Imports a parser that converts model output into readable text.

Part 1c — LCEL

LangChain:

A framework that helps structure how applications interact with LLMs instead of writing raw API calls.

Cell 10

Setup cell preparing tools for chaining.

Cell 11

Shows step-by-step manual prompt formatting.

Cell 12

Creates a reusable chain that connects prompt → model → parser.

The notebook teaches how to connect a language model using LangChain, control its behavior with prompts and temperature, reuse prompts with templates, clean outputs using parsers, and build pipelines using LCEL. These tools make LLM applications structured, reusable, and easier to manage.

Assignment :

Assignment

Create a chain that:

1. Takes a movie name.
2. Asks for its release year.
3. Calculates how many years ago that was (You can try just asking the LLM to do the math).

Try to do it in **one line of LCEL**.

```
movie_prompt = ChatPromptTemplate.from_template(
    """
    Movie: {movie}

    1) Find the release year.
    2) Calculate how many years ago it was released (assume current year is 2025).
    3) Respond clearly in one sentence.
    """
)
```

```
movie_chain = movie_prompt | llm | StrOutputParser()
movie_chain.invoke({"movie": "Inception"})
```

```
'Inception was released in 2010, making it 15 years old as of 2025.'
```

This code creates a LangChain pipeline that takes a movie name as input and asks the LLM to find its release year and calculate how many years ago it was released, assuming the current year is 2025. The prompt template makes the input reusable, and the LCEL chain connects the prompt, the language model, and an output parser in one step. When the chain is invoked with a movie name, the model performs the reasoning and returns the answer in a single clear sentence.

Part2: Prompt Engineering

Part 2a:

Cell 1

This cell installs the required libraries so LangChain and the Gemini model can be used in the notebook. Loads environment variables like the API key securely, so sensitive information isn't written directly in the code. Imports the necessary LangChain and model classes that will be used to build prompts and interact with the LLM. Creates the language model object. This is where model settings like temperature are defined.

Cell 2

Tests the model with a simple prompt to confirm that everything is working correctly.

Cell 3

Defines a structured prompt so instructions to the model are clearer and reusable.

Assignment :

Assignment

Write a structured prompt to generate a **Python Function**.

- **Context:** You are a Senior Python Dev.
- **Objective:** Write a function to reverse a string.
- **Constraint:** It must use recursion (no slicing `[::-1]`).
- **Style:** Include detailed docstrings.

[4] ✓ 0s

```
structured_prompt = """
Context:
You are a Senior Python Developer.

Objective:
Write a Python function that reverses a string.

Constraints:
- You must use recursion.
- Do NOT use slicing (like [::-1]).

Style:
- Include detailed docstrings explaining the function.
- Keep the code clean and readable.
"""
```

[5] ✓ 17s

```
response = llm.invoke(structured_prompt)
print(response.content)
```

[5]
17s

```python
response = llm.invoke(structured_prompt)
print(response.content)
```

```python
def reverse_string_recursive(s: str) -> str:
    """
    Reverses a given string using a recursive approach.

    This function breaks down the problem by taking the first character
    of the string and appending it to the end of the recursively reversed
    remainder of the string. This process continues until the string is
    empty or contains only one character, which serves as the base case.

    Args:
        s (str): The input string to be reversed.

    Returns:
        str: The reversed string.

    Constraints:
        - **Recursion**: The function calls itself to solve smaller subproblems.
        - **No Slicing (like [::-1])**: The specific string reversal slice `[::-1]`
          is explicitly avoided. Standard slicing like `s[1:]` (to get the
          rest of the string) and `s[0]` (to get the first character) are
          used as they are fundamental for recursive string decomposition
          and are not considered the "reversal slice" forbidden by the constraint.

    Examples:
        >>> reverse_string_recursive("hello")
        'olleh'
        >>> reverse_string_recursive("Python")
        'nohtyP'
        >>> reverse_string_recursive("")
        ''
        >>> reverse_string_recursive("a")
        'a'
        >>> reverse_string_recursive("racecar")
        'racecar'
        >>> reverse_string_recursive("12345")
        '54321'
    """
    # Base Case:
    # If the string is empty or contains only one character, it is already
    # its own reverse. This is the stopping condition for the recursion.
    if len(s) <= 1:
        return s
    else:
        # Recursive Step:
        # 1. Take the 'rest of the string' (s[1:]) and recursively reverse it.
```

```python
    """
    # Base Case:
    # If the string is empty or contains only one character, it is already
    # its own reverse. This is the stopping condition for the recursion.
    if len(s) <= 1:
        return s
    else:
        # Recursive Step:
        # 1. Take the 'rest of the string' (s[1:]) and recursively reverse it.
        #    This means calling reverse_string_recursive on a smaller substring.
        # 2. Take the 'first character' (s[0]) of the original string.
        # 3. Concatenate the result of step 1 with the character from step 2.
        #    This effectively moves the first character to the end of the
        #    already reversed remainder.

        # Example Trace for "abc":
        # reverse_string_recursive("abc")
        #   -> reverse_string_recursive("bc") + 'a'
        #      -> reverse_string_recursive("c") + 'b'
        #         -> 'c' (base case)
        #      -> 'c' + 'b' -> "cb"
        #   -> "cb" + 'a' -> "cba"
        return reverse_string_recursive(s[1:]) + s[0]
```

This assignment is about writing a structured prompt to generate Python code. First, we clearly define the Context — we tell the AI to act like a Senior Python Developer. Then we give the Objective — write a function that reverses a string. After that, we add Constraints:

- The function must use recursion
- We cannot use slicing like [::-1]

Finally, we mention the Style requirements:

- Add detailed docstrings
- Keep the code clean and readable

So basically, this assignment teaches us how to give clear and structured instructions to an AI so it generates the correct Python function.

Part 2b: Zero-shot to few-shot

Cell 4

Attention mechanism

Cell 5

Zero-Shot learning model output example

Cell 6

Few-shot learning model output example

Part 2c: Advanced templates and theory

Cell 7

Setting up for prompt testing

Cell 8

Dynamic few shotting example

Part3: Advanced Prompting

Part3a:

Cell 1

This cell installs required libraries, loads environment variables, and initializes the language model so we can run experiments.

Cell 2

Runs the math question using a normal prompt. This demonstrates how the model may give incorrect answers without structured reasoning.

Cell 3

Uses a Chain of Thought prompt ("think step by step"). This forces the model to show reasoning and improves accuracy.

Part3b: tree of thoughts and graph of thoughts

Cell 4

Repeats environment setup to ensure everything is ready for advanced prompting experiments.

Cell 5

Implements a ToT-style workflow using LangChain components to generate and evaluate multiple reasoning branches.

Cell 6

Demonstrates GoT-style branching using parallel prompt generation and aggregation.

Part 4 :RAG and Vector Stores

Part 4a: embeddings and vector space

Cell 1

This cell installs required libraries and loads environment variables. Then it loads a Hugging Face embedding model. This model converts text into numerical vectors so that the computer can understand meaning.

Cell 2

Converts words to vector

Cell 3

Computing the Cosine Similarity

Part 4b: Naïve RAG pipeline

Cell 4

This installs more libraries including FAISS and Google Gemini. It loads Google API key, Gemini model (gemini-2.5-flash) and the same embedding model again. This prepares everything for building a RAG system.

Cell 5

Create documents to store vector database.

Cell 6

We store the documents inside **FAISS**, which is a vector database. Then we create a retriever to search similar documents based on a question.

Cell 7

The retriever finds the correct document, and the LLM answers using only that context. This is a simple RAG pipeline.

Part 4c: deep dive into indexing algorithms

Cell 8

This is mock data to test different FAISS indexing methods.

Cell 9

We create a **Flat L2 index**. This checks every vector when searching. It is accurate but slow for large data.

Cell 10

We create an IVF (Inverted File) index. It divides vectors into clusters and searches only inside relevant clusters. This makes search faster than flat index.

Cell 11

We create an HNSW index. This builds a graph structure for fast searching. It is very efficient and commonly used in large systems.

Cell 12

We create a Product Quantization (PQ) index. It compresses vectors into smaller parts to reduce memory usage. This saves RAM but slightly reduces accuracy.